

Taylor & Francis
Taylor & Francis Group

Journal of Computational and Graphical Statistics

ISSN: 1061-8600 (Print) 1537-2715 (Online) Journal homepage: www.tandfonline.com/journals/ucgs20

Learning Multiple Quantiles With Neural Networks

Sang Jun Moon, Jong-June Jeon, Jason Sang Hun Lee & Yongdai Kim

To cite this article: Sang Jun Moon, Jong-June Jeon, Jason Sang Hun Lee & Yongdai Kim (2021) Learning Multiple Quantiles With Neural Networks, Journal of Computational and Graphical Statistics, 30:4, 1238-1248, DOI: 10.1080/10618600.2021.1909601

To link to this article: https://doi.org/10.1080/10618600.2021.1909601

© 2021 The Author(s). Published with license by Taylor & Francis Group, LLC

View supplementary material

Published online: 07 May 2021.

Submit your article to this journal

Article views: 8978

View related articles

View Crossmark data

Citing articles: 26 View citing articles

Full Terms & Conditions of access and use can be found at
https://www.tandfonline.com/action/journalInformation?journalCode=ucgs20

JOURNAL OF COMPUTATIONAL AND GRAPHICAL STATISTICS
2021, VOL. 30, NO. 4, 1238–1248
https://doi.org/10.1080/10618600.2021.1909601

Taylor & Francis
Taylor & Francis Group

OPEN ACCESS

Check for updates

Learning Multiple Quantiles With Neural Networks

Sang Jun Moon[a], Jong-June Jeon[a], Jason Sang Hun Lee[b], and Yongdai Kim[c]

[a]Department of Statistics, University of Seoul, Seoul, South Korea; [b]Department of Physics, University of Seoul, Seoul, South Korea; [c]Department of Statistics, Seoul National University, Seoul, South Korea

ABSTRACT

We present a neural network model for estimation of multiple conditional quantiles that satisfies the noncrossing property. Motivated by linear noncrossing quantile regression, we propose a noncrossing quantile neural network model with inequality constraints. In particular, to use the first-order optimization method, we develop a new algorithm for fitting the proposed model. This algorithm gives a nearly optimal solution without the projected gradient step that requires polynomial computation time. We compare the performance of our proposed model with that of existing neural network models on simulated and real precipitation data. Supplementary materials for this article are available online.

ARTICLE HISTORY

Received September 2019
Revised December 2020

KEYWORDS

Deep learning; Feed-forward neural network; Interior point algorithm; Projected gradient algorithm; Quantile regression

1. Introduction

The quantile regression of Koenker and Bassett Jr (1978) is popularly used as an alternative to conventional regression to learn the relationship between response and predictor variables. An important advantage of quantile regression is that it allows one to obtain informative summaries of the conditional distribution of the response variable for given predictor variables by estimating multiple quantiles simultaneously (Koenker 2004; Li and Zhu 2008).

The linear quantile regression model is widely used because of its simple structure and well-established theoretical properties (Koenker 2004; Li and Zhu 2008). However, the assumptions built into linear quantile regression models are often inadequate to describe data comprised of complex structures. Numerous nonlinear models have been proposed for quantile regression. Hall, Wolff, and Yao (1999) and Yu and Jones (1998) used a local polynomial regression; Doksum and Koo (2000), Koenker, Ng, and Portnoy (1994), and Silverman (1985) used a spline basis function; Hwang and Shim (2005) and Sohn et al. (2008) proposed the support vector regression (SVR) model for quantile regression; Taylor (2000) and Feng et al. (2010) considered a quantile regression neural network (QRNN); and Kai, Li, and Zou (2010), Xu et al. (2017), and Hatalis et al. (2017) proposed composite quantile regression for simultaneous estimation of multiple quantiles.

When multiple quantiles are estimated simultaneously, we often suffer from the so-called crossing problem. The crossing problem is the phenomenon that occurs when the estimated quantiles violate the basic property that the cumulative distribution function be nondecreasing. For example, it is difficult to accept that the estimated 95% quantile is smaller than the estimated 90% one (Xu et al. 2017). In particular, the crossing problem frequently arises when nonlinear models such as SVR and neural network models are used.

To resolve the crossing problem, noncrossing constraints are added when multiple quantiles are estimated. There are two approaches constraining the crossing problem. The first approach is to transform the parameters in the model to satisfy the noncrossing constraint. Cannon (2018) employed such a transformation to construct monotone increasing bias parameters across quantiles in the neural network model. Meinshausen (2006) and Gasthaus et al. (2019) also used monotone transformation for positive weight parameters associated with quantile functions.

In contrast, the second approach is to explicitly constrain the model space to noncrossing quantiles. Some examples of this approach include the noncrossing SVR by Takeuchi et al. (2006) and Sangnier, Fercoq, and d'Alché Buc (2016) and the noncrossing spline regression by Bondell, Reich, and Wang (2010). Due to the constraint, there is no need to use a monotonically increasing basis function or positive coefficients in these models.

Between these two approaches, the second one is more promising for large-scale data, because various constraint optimization algorithms, such as the projected gradient algorithm or the interior point algorithm, can be used to estimate multiple quantiles simultaneously with minor modifications. However, computational costs can be very large and there is a possibility that these algorithms will not converge. Hence, we still need to develop an efficient optimization algorithm that specializes in estimating multiple quantiles, which is the main theme of this paper.

The purpose of this article is to propose a noncrossing quantile regression model with neural networks and to develop an efficient optimization algorithm to fit the model. The proposed model is based on the idea of the noncrossing SVR (Takeuchi and Furuhashi 2004). We apply the inequality constraints used in the noncrossing SVR to the hidden layer of a given neural

CONTACT Jong-June Jeon jj.jeon@gmail.com Department of Statistics, University of Seoul, Seoul, South Korea.
Supplementary materials for this article are available online. Please go to www.tandfonline.com/r/JCGS.
© 2021 The Author(s). Published with license by Taylor & Francis Group, LLC.
This is an Open Access article distributed under the terms of the Creative Commons Attribution-NonCommercial-NoDerivatives License (http://creativecommons.org/licenses/by-nc-nd/4.0/), which permits non-commercial re-use, distribution, and reproduction in any medium, provided the original work is properly cited, and is not altered, transformed, or built upon in any way.

network. To train the network, we develop an optimization algorithm that only requires the gradient of the objective function with a l_1-penalty. As a result, our proposed algorithm is easy to implement using well-developed open-source software for deep neural networks.

This article is organized as follows. Section 2 introduces the neural network for quantile regression and proposes a new noncrossing multiple quantiles regression model. Section 3 investigates the first-order optimization methods used to train the proposed model. Section 4 explains our proposed computation algorithm and Section 5 shows the results of the numerical simulation and real data analysis. Finally, our conclusions and discussion follow in Section 6.

2. Quantile Regression Model

Let $(y_i, \mathbf{X}_i)$ for $i = 1, \ldots, n$ be pairs of the response variable and the predictor vector, which are assumed be independent realizations of random vector $(Y, \mathbf{X})$ for $Y \in \mathbb{R}$ and $\mathbf{X} \in \mathbb{R}^p$. Let $F(\cdot|\mathbf{X})$ be the conditional cumulative distribution function of Y given $\mathbf{X}$. For a given $\tau \in (0,1)$, the conditional τ-quantile function $f_\tau(\mathbf{X})$ is defined as $\inf\{y \in \mathbb{R} : F(y|\mathbf{X} = \mathbf{X}) \geq \tau\}$.

Koenker, Ng, and Portnoy (1994) considered a linear model for the τ-quantile function: $f_\tau(\mathbf{X}) = \mathbf{X}^\top \boldsymbol{\beta}$ for $\boldsymbol{\beta} = (\beta_1, \ldots, \beta_p)^\top \in \mathbb{R}^p$, and propose an estimation method for $\boldsymbol{\beta}$ that minimizes the quantile risk $n^{-1}\sum_{i=1}^n \rho_\tau(y_i - \mathbf{X}_i^\top \boldsymbol{\beta})$, where $\rho_\tau(u) = u(\tau - I(u < 0))$. Simultaneous estimation of multiple quantiles was proposed by Zou et al. (2008) and Zou and Yuan (2008b). For a given set of quantiles $0 < \tau_1 < \cdots < \tau_K < 1$, the conditional τ_k-quantile function, $f_{\tau_k}(\mathbf{X})$, is assumed as $f_{\tau_k}(\mathbf{X}) = \mathbf{X}^\top \boldsymbol{\beta}_k$ for $\boldsymbol{\beta}_k = (\beta_{k1}, \ldots, \beta_{kp})^\top \in \mathbb{R}^p$ and $k = 1, \ldots, K$. And, $(\boldsymbol{\beta}_1, \ldots, \boldsymbol{\beta}_K)$ is estimated by minimizing the composite quantile risk $(nK)^{-1}\sum_{k=1}^K \sum_{i=1}^n \rho_{\tau_k}(y_i - \mathbf{X}_i^\top \boldsymbol{\beta}_k)$. For convenience, we refer to the composite quantile risk as simply quantile risk.

By the definition of quantiles, the conditional quantile functions should not be crossing. That is, $f_{\tau_j}(\mathbf{X}) \leq f_{\tau_k}(\mathbf{X})$ should be satisfied for all $\mathbf{X}$ when $\tau_j \leq \tau_k$. The noncrossing condition of the quantile functions is equivalent to $\mathbf{X}^\top \boldsymbol{\beta}_1 \leq \cdots \leq \mathbf{X}^\top \boldsymbol{\beta}_K$ for all $\mathbf{X}$ if $f_{\tau_k}(\mathbf{X}) = \mathbf{X}^\top \boldsymbol{\beta}_k$. Therefore, to estimate the linear quantile functions which are not crossing, we solve the following constrained optimization problem.

$$\begin{aligned} \min_{\beta_1,\ldots,\beta_K \in \mathbb{R}^p} \quad & \frac{1}{nK}\sum_{k=1}^K \sum_{i=1}^n \rho_{\tau_k}(y_i - \mathbf{X}_i^\top \boldsymbol{\beta}_k) \\ \text{subject to} \quad & \mathbf{X}^\top \boldsymbol{\beta}_1 \leq \cdots \leq \mathbf{X}^\top \boldsymbol{\beta}_K \text{ for all } \mathbf{X} \in \mathbb{R}^p. \end{aligned}$$

Extending the above approach to estimate noncrossing quantiles in nonlinear models, we can consider the following models: $f_{\tau_k}(\mathbf{X}) = \mathbf{z}(\mathbf{X};\boldsymbol{\theta})^\top \boldsymbol{\beta}_k$ for $k = 1, \ldots, K$, where $\boldsymbol{\beta}_k \in \mathbb{R}^q$ and $\mathbf{z}(\cdot;\boldsymbol{\theta})$ is a feature map from $\mathbb{R}^p$ to $\{1\} \times \mathbb{R}^{q-1}$ parameterized by $\boldsymbol{\theta} \in \Theta$. Here, the first element of $\mathbf{z}(\mathbf{X};\boldsymbol{\theta})$ is 1, each β_{k1} is a bias parameter, and Θ is the parameter space of $\boldsymbol{\theta}$. We can estimate $\boldsymbol{\theta}$ as well as $\boldsymbol{\beta}_k$s by solving

$$\begin{aligned} \min_{\beta_1,\ldots,\beta_K \in \mathbb{R}^q, \theta \in \Theta} \quad & \frac{1}{nK}\sum_{k=1}^K \sum_{i=1}^n \rho_{\tau_k}(y_i - \mathbf{z}(\mathbf{X}_i;\boldsymbol{\theta})^\top \boldsymbol{\beta}_k) \qquad (1) \\ \text{subject to} \quad & \mathbf{z}(\mathbf{X};\boldsymbol{\theta})^\top \boldsymbol{\beta}_{k-1} \leq \mathbf{z}(\mathbf{X};\boldsymbol{\theta})^\top \boldsymbol{\beta}_k \\ & \text{for all } \mathbf{X} \in \mathbb{R}^p,\ k = 2, \ldots, K. \end{aligned}$$

When $\mathbf{z}(\mathbf{X};\boldsymbol{\theta}) \in \{1\} \times [0,1]^{q-1}$ for all $\mathbf{X} \in \mathbb{R}^p$ and $\boldsymbol{\theta} \in \Theta$, problem (1) is equivalent to the following simpler form:

$$\begin{aligned} \min_{\beta_1,\cdots\beta_K \in \mathbb{R}^q, \theta \in \Theta} \quad & \frac{1}{nK}\sum_{k=1}^K \sum_{i=1}^n \rho_{\tau_k}(y_i - \mathbf{z}(\mathbf{X}_i;\boldsymbol{\theta})^\top \boldsymbol{\beta}_k) \qquad (2) \\ \text{subject to} \quad & \mathbf{v}^\top \boldsymbol{\beta}_{k-1} \leq \mathbf{v}^\top \boldsymbol{\beta}_k \\ & \text{for all } \mathbf{v} \in \{1\} \times \{0,1\}^{q-1},\ k = 2, \ldots, K. \end{aligned}$$

This is because the feasible set of Equation (1),

$$\{(\boldsymbol{\theta}, \boldsymbol{\beta}) \in \Theta \times \mathbb{R}^{qK} : \mathbf{z}(\mathbf{X};\boldsymbol{\theta})^\top \boldsymbol{\beta}_{k-1} \leq \mathbf{z}(\mathbf{X};\boldsymbol{\theta})^\top \boldsymbol{\beta}_k \text{ for all } \mathbf{X},\ k = 2, \ldots, K\},$$

is the same as the feasible set of Equation (2),

$$\{(\boldsymbol{\theta}, \boldsymbol{\beta}) \in \Theta \times \mathbb{R}^{qK} : \mathbf{v}^\top \boldsymbol{\beta}_{k-1} \leq \mathbf{v}^\top \boldsymbol{\beta}_k \text{ for all } \mathbf{v} \in \{1\} \times \{0,1\}^{q-1},\ k = 2, \ldots, K\}, \qquad (3)$$

when $\mathbf{z}(\mathbf{X};\boldsymbol{\theta}) \in \{1\} \times [0,1]^{q-1}$ for all $\mathbf{X} \in \mathbb{R}^p$ and $\boldsymbol{\theta} \in \Theta$ (Bondell, Reich, and Wang 2010). Note that the constraints in Equation (2) do not depend on $\boldsymbol{\theta}$ and thus problem (2) is much easier than problem (1).

However, the feasible set (3) is a polyhedron consisting of the intersection of $2^{q-1}(K-1)$ half spaces induced by the linear inequalities; thus, solving Equation (2) is still intractable when q is large. Bondell, Reich, and Wang (2010) proposed an efficient method of simplifying problem (2) further by reparameterizing $\boldsymbol{\beta}_k$ as $\boldsymbol{\delta}_k$ for $k = 1, \ldots, K$, where

$$\boldsymbol{\delta}_1 = \boldsymbol{\beta}_1,\ \boldsymbol{\delta}_k = \boldsymbol{\beta}_k - \boldsymbol{\beta}_{k-1},\ k = 2, \ldots, K. \qquad (4)$$

The following proposition proves that $\mathbf{v}^\top \boldsymbol{\beta}_{k-1} \leq \mathbf{v}^\top \boldsymbol{\beta}_k$ for all $\mathbf{v} \in \{1\} \times \{0,1\}^{q-1}$ if and only if $\delta_{k1} - \sum_{j=2}^q \max(0, -\delta_{kj}) \geq 0$ for $k = 2, \ldots, K$, where δ_{kj} is the jth element of $\boldsymbol{\delta}_k$.

Proposition 1. Let $\boldsymbol{\beta} = (\boldsymbol{\beta}_1, \ldots, \boldsymbol{\beta}_K)$, $\boldsymbol{\delta} = (\boldsymbol{\delta}_1, \ldots, \boldsymbol{\delta}_K)$ and $\boldsymbol{\beta}_k, \boldsymbol{\delta}_k \in \mathbb{R}^q$ for $k = 1, \ldots, K$. Define

$$\mathcal{C}_b = \left\{\boldsymbol{\beta} \in \mathbb{R}^{qK} : \mathbf{v}^\top \boldsymbol{\beta}_{k-1} \leq \mathbf{v}^\top \boldsymbol{\beta}_k \text{ for all } \mathbf{v} \in \{1\} \times \{0,1\}^{q-1}, \text{ for } k = 2, \ldots, K\right\}$$

and

$$\mathcal{C}_d = \left\{\boldsymbol{\delta} \in \mathbb{R}^{qK} : \delta_{k1} - \sum_{j=2}^q \max(0, -\delta_{kj}) \geq 0, \text{ for } k = 2, \ldots, K\right\}.$$

Then,

$$\mathcal{C}_b = \left\{\boldsymbol{\beta} \in \mathbb{R}^{qK} : \boldsymbol{\beta}_1 = \boldsymbol{\delta}_1, \boldsymbol{\beta}_k - \boldsymbol{\beta}_{k-1} = \boldsymbol{\delta}_k, \text{ for } k = 2, \ldots, K, \text{ and } \boldsymbol{\delta} \in \mathcal{C}_d\right\}.$$

The proof is appended in the supplementary material. As the mapping from $\mathcal{C}_b$ to $\mathcal{C}_d$ in Equation (4) is bijective, Proposition 1 implies that problem (2) is equivalent to the following problem:

$$\begin{aligned} \min_{\delta \in \mathbb{R}^{qK}, \theta \in \Theta} \quad & L(\boldsymbol{\delta}, \boldsymbol{\theta}) \qquad (5) \\ \text{subject to} \quad & \boldsymbol{\delta} \in \mathcal{C}_d, \end{aligned}$$

where $L(\boldsymbol{\delta},\boldsymbol{\theta}) = (nK)^{-1}\sum_{k=1}^{K}\sum_{i=1}^{n}\rho_{\tau_k}\left(y_i - \sum_{l=1}^{k}\mathbf{z}(\mathbf{X}_i;\boldsymbol{\theta})^\top\boldsymbol{\delta}_l\right)$ and $\mathcal{C}_d$ is defined in Proposition 1. That is, the solution of Equation (2) is obtained by $\boldsymbol{\beta}_k = \sum_{l=1}^{k}\boldsymbol{\delta}_l$ for $k = 1,\ldots,K$ whenever $\boldsymbol{\delta}$ is the solution of Equation (5).

We model $\mathbf{z}(\cdot;\boldsymbol{\theta})$ by a deep neural network and propose an efficient algorithm to solve Equation (5). We call the considered model,

$$f_{\tau_k}(\mathbf{X}) = \sum_{l=1}^{k}\mathbf{z}(\mathbf{X};\boldsymbol{\theta})^\top\boldsymbol{\delta}_l \text{ for } k = 1,\ldots,K$$
$$\text{subject to} \quad \boldsymbol{\delta} \in \mathcal{C}_d$$

for the target quantiles $(\tau_1,\ldots,\tau_K)$, the noncrossing multiple quantile regression with neural networks (NMQN) model.

The condition $\mathbf{z}(\mathbf{X};\boldsymbol{\theta}) \in \{1\}\times[0,1]^{q-1}$ for all $\mathbf{X}\in\mathbb{R}^p$ and $\boldsymbol{\theta}\in\Theta$ is necessary to solve (1) with problem (5). To ensure this condition, we use the sigmoid activation function at the top of the hidden layers of $\mathbf{z}(\cdot;\boldsymbol{\theta})$. Note that the feasibility condition of (5) is equivalent to the noncrossing condition of our quantile regression model.

3. Using Off-the-Shelf Optimization Algorithms

The main theme of this section is not to develop a new algorithm but to explain the limitations of off-the-shelf algorithms to solve Equation (5) for fitting the NMQN. We focus on two gradient-based algorithms, the projected gradient algorithm and the interior point algorithm.

3.1. Projected Gradient Algorithm

The projected gradient algorithm solves constrained optimization problems using the gradient descent approach. It consists of two steps: first, to update the current solution using a standard gradient descent method; second, to project the updated solution to the feasible set associated with the constraints. For convex optimization problems, the convergence of the projected gradient algorithm has been proven by Alber, Iusem, and Solodov (1998) and Iusem (2003). Additionally, the projected gradient algorithm is popularly used to train neural network models with constraints (Xia 2004; Chorowski and Zurada 2014; Pathak, Krahenbuhl, and Darrell 2015).

We will explain how to employ the projected gradient algorithm to solve Equation (5). Let $\nabla_\delta L(\boldsymbol{\delta},\boldsymbol{\theta})$ and $\nabla_\theta L(\boldsymbol{\delta},\boldsymbol{\theta})$ be the gradients of $L(\boldsymbol{\delta},\boldsymbol{\theta})$ with respect to $\boldsymbol{\delta}$ and $\boldsymbol{\beta}$, respectively. Even though $L(\boldsymbol{\delta},\boldsymbol{\theta})$ is non-differentiable, we use the terms $\nabla_\delta L(\boldsymbol{\delta},\boldsymbol{\theta})$ and $\nabla_\theta L(\boldsymbol{\delta},\boldsymbol{\theta})$ as subdifferentials of $L(\boldsymbol{\delta},\boldsymbol{\theta})$ with respect to $\boldsymbol{\delta}$ and $\boldsymbol{\theta}$ throughout this paper. We solve Equation (5) using the projected gradient algorithm, which iterates the following two steps until the solution converges: 1) update the current solution $(\boldsymbol{\delta}^{(t)},\boldsymbol{\theta}^{(t)})$ by $\bar{\boldsymbol{\delta}} = \boldsymbol{\delta}^{(t)} - \gamma_t\nabla_\delta L(\boldsymbol{\delta}^{(t)},\boldsymbol{\theta}^{(t)})$ and $\bar{\boldsymbol{\theta}} = \boldsymbol{\theta}^{(t)} - \gamma_t\nabla_\theta L(\boldsymbol{\delta}^{(t)},\boldsymbol{\theta}^{(t)})$, where γ_t is a learning rate, 2) project the current solution to the feasible set $\mathcal{C}_d$ to have

$$(\boldsymbol{\delta}^{(t+1)},\boldsymbol{\theta}^{(t+1)}) = (\Pi_{\mathcal{C}_d}(\bar{\boldsymbol{\delta}}),\bar{\boldsymbol{\theta}}), \tag{6}$$

where $\Pi_{\mathcal{C}_d}$ is the projection operator from $\mathbb{R}^{qK}$ to $\mathcal{C}_d$.

A computational bottleneck of the projected gradient algorithm is obtaining $\Pi_{\mathcal{C}_d}(\bar{\boldsymbol{\delta}})$, which is the solution of

$$\min_{\delta\in\mathbb{R}^{qK}} \|\boldsymbol{\delta} - \bar{\boldsymbol{\delta}}\|_2^2 \quad \text{subject to} \quad \boldsymbol{\delta}\in\mathcal{C}_d. \tag{7}$$

Since the objective function in Equation (7) is quadratic and $\mathcal{C}_d$ is a convex polyhedron induced by linear constraints, quadratic programming can be applied to solve Equation (7). Note that the quadratic programming requires significant computations to solve Equation (7), and thus the projected gradient algorithm is not easily scalable because it uses the quadratic programming iteratively.

3.2. Interior Point Algorithm

The interior point algorithm obtains the solution of Equation (5) by solving the following problem:

$$\min_{\delta\in\mathbb{R}^{qK},\theta\in\Theta} L(\boldsymbol{\delta},\boldsymbol{\theta}) + B(\boldsymbol{\delta};M), \tag{8}$$

where $B(\boldsymbol{\delta};M)$ is the log barrier function which is defined as

$$B(\boldsymbol{\delta};M) = -\frac{1}{M}\sum_{k=2}^{K}\log\left(\delta_{k1} - \sum_{j=2}^{q}\max(0,-\delta_{kj}) + \epsilon\right),\ \epsilon\geq 0.$$

In the barrier function, M is a positive tuning parameter and ϵ is a small nonnegative constant used to ensure numerical stability of the algorithm.

Since Equation (8) is an unconstrained optimization problem, the interior point algorithm can use the standard gradient descent method, which updates the solution by

$$(\boldsymbol{\delta}^{(t+1)},\boldsymbol{\theta}^{(t+1)}) = (\boldsymbol{\delta}^{(t)},\boldsymbol{\theta}^{(t)}) - \gamma_t\left(\nabla L(\boldsymbol{\delta}^{(t)},\boldsymbol{\theta}^{(t)}) + \nabla B(\boldsymbol{\delta}^{(t)};M)\right). \tag{9}$$

However, this update Equation (9) may not work unless the learning rate γ_t is carefully selected. For example, $\delta_{k1}^{(t+1)} - \sum_{j=2}^{q}\max(0,-\delta_{kj}^{(t+1)})$ can be less than $-\epsilon$ for some k when the learning rate γ_t is too large. In this case, the interior point algorithm is aborted. Moreover, selecting an appropriate γ_t would be difficult and computationally demanding, especially when K is large.

4. Proposed Algorithm

We consider two requirements when designing a new algorithm to solve Equation (5). First, the algorithm can be implemented by the standard gradient descent method; Second, the selection of parameters associated with the algorithm is minimally required in the middle of updating solutions. We propose a gradient descent algorithm with the l_1-penalty function to solve Equation (5). We explain the motivation behind the proposed algorithm first and present the properties of the solution obtained by the algorithm. Finally, we investigate the noncrossing property of the estimated NMQN by the proposed algorithm.

4.1. Motivation

The main problem with the interior point algorithm is that $B(\boldsymbol{\delta}; M)$ is not defined out of $\mathcal{C}_d$. Typically, a feasible $\boldsymbol{\delta}^{(t+1)}$ can be chosen by proper selection of the learning rate γ_t and M for each step. But we can avoid the problem without correcting γ_t and M. By simply updating $\delta_{k1}^{(t+1)}$ as $\max\left(\delta_{k1}^{(t+1)}, \sum_{j=2}^{q} \max(0, -\delta_{kj}^{(t+1)})\right)$ after computing Equation (9) at the tth step, we can obtain another feasible $\boldsymbol{\delta}^{(t+1)}$. Obviously, $\delta_{k1}^{(t+1)} = \max\left(\delta_{k1}^{(t+1)}, \sum_{j=2}^{q} \max(0, -\delta_{kj}^{(t+1)})\right)$ for $k = 2, \ldots, K$ if and only if $\boldsymbol{\delta}^{(t+1)} \in \mathcal{C}_d$, which implies that the corrected $\boldsymbol{\delta}^{(t+1)}$ is always contained in $\mathcal{C}_d$ and there is no abortion of the interior point algorithm for any γ_t and M. Given this simple correction, we can rewrite problem (8) with the equality constraint leading to the feasibility condition of $\boldsymbol{\delta}$, as follows:

$$\min_{\delta \in \mathbb{R}^{qK}, \theta \in \Theta} \quad L(\boldsymbol{\delta}, \boldsymbol{\theta}) + B(\boldsymbol{\delta}; M) \tag{10}$$

$$\text{subject to} \quad \delta_{k1} = \max\left(\delta_{k1}, \sum_{j=2}^{q} \max(0, -\delta_{kj})\right) \text{ for } k = 2, \ldots, K.$$

To solve (10) by the gradient descent method, we replace δ_{k1} in the log barrier function $B(\boldsymbol{\delta}; M)$ with $\max(\delta_{k1}, \sum_{j=2}^{q} \max(0, -\delta_{kj}))$ for $k = 2, \ldots, K$ and use the l_1-penalty function to shrink the difference between δ_{k1} and $\max(\delta_{k1}, \sum_{j=2}^{q} \max(0, -\delta_{kj}))$. We denote the barrier and penalty functions as

$$\phi(\boldsymbol{\delta}_k; M) = -\frac{1}{M} \log\left(\max\left(\delta_{k1}, \sum_{j=2}^{q} \max(0, -\delta_{kj})\right) - \sum_{j=2}^{q} \max(0, -\delta_{kj}) + \epsilon\right)$$

and

$$J(\boldsymbol{\delta}_k) = \left|\delta_{k1} - \max\left(\delta_{k1}, \sum_{j=2}^{q} \max(0, -\delta_{kj})\right)\right|,$$

respectively. Finally, we define a new problem to solve Equation (5) as

$$\min_{\delta \in \mathbb{R}^{qK}, \theta \in \Theta} \quad L(\boldsymbol{\delta}, \boldsymbol{\theta}) + \lambda \sum_{k=2}^{K} J(\boldsymbol{\delta}_k) + \sum_{k=2}^{K} \phi(\boldsymbol{\delta}_k; M), \tag{11}$$

where $\lambda > 0$ is a tuning parameter. Since the gradient of $\phi(\boldsymbol{\delta}_k; M)$ is always defined, we can apply the gradient descent method to solve Equation (11) without being concerned with abortion. Note that the solution of Equation (11) is not always equal to that of (8), because it depends on λ. However, we can show the condition for λ for which the solutions of Equations (8) and (11) are equal, as discussed in the next subsection.

4.2. l_1-penalization Algorithm

The l_1-penalty function plays a key role in solving Equation (11) by the gradient descent method, especially when a current solution is not contained in $\mathcal{C}_d$. Denote the current solution at the tth step by $\boldsymbol{\delta}^{(t)}$ and suppose that $\delta_{k1}^{(t)} < \sum_{j=2}^{q} \max(0, -\delta_{kj}^{(t)})$ and $\delta_{kj}^{(t)} \neq 0$ for all j. Clearly, the l_1-penalty imposes a penalty on the objective function for violations of the feasibility condition, because $J(\boldsymbol{\delta}_k)$ has a positive value only when $\delta_{k1} < \sum_{j=2}^{q} \max(0, -\delta_{kj})$. In addition, it is apparent that $\nabla_{\delta_k} J(\boldsymbol{\delta}_k^{(t)})$ is orthogonal to the subspace $\{\boldsymbol{\delta}_k \in \mathbb{R}^q : \delta_{k1} + \sum_{j:\delta_{kj}^{(t)}<0, j\geq 2} \delta_{kj} = 0\}$. Since $-\nabla_{\delta_k} J(\boldsymbol{\delta}_k^{(t)})$ forces the direction of the current solution onto $\{\boldsymbol{\delta}_k \in \mathbb{R}^q : \delta_{k1} + \sum_{j:\delta_{kj}^{(t)}<0, j\geq 2} \delta_{kj} \geq 0\}$, we can update the current solution to include the consideration of the feasibility condition.

The barrier function plays no role in updating $\boldsymbol{\delta}_k^{(t)}$ because $\nabla_{\delta_k} \phi(\boldsymbol{\delta}_k^{(t)}; M) = 0$ for $\boldsymbol{\delta}_k^{(t)} \notin \mathcal{C}_d$. For $\boldsymbol{\delta} \in \mathcal{C}_d$, the gradient of the barrier function is negligible when M is large. This means that the barrier function has little effect on the final solution using the gradient descent method. Therefore, we can solve Equation (10) without the barrier function to obtain the solution to Equation (5), which is described in Algorithm 1. We call this proposed method of solving Equation (12) the l_1-penalization algorithm.

$$\min_{\delta \in \mathbb{R}^{qK}, \theta \in \Theta} \quad L(\boldsymbol{\delta}, \boldsymbol{\theta}) + \lambda \sum_{k=2}^{K} J(\boldsymbol{\delta}_k). \tag{12}$$

Algorithm 1 l_1-penalization algorithm

Input: $\mathbf{X} \in \mathbb{R}^{n \times p}$, $Y \in \mathbb{R}^n$, $(\tau_1, \ldots, \tau_K)$
Output: $\boldsymbol{\theta}$, $\boldsymbol{\delta}$
Set: T, γ_t, λ
Initialize: $t = 1$, $\boldsymbol{\theta}^{(t)}$ and $\boldsymbol{\delta}^{(t)}$
for $t = 1$ to T **do**
  Update $\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \gamma_t \nabla_\theta L(\boldsymbol{\delta}^{(t)}, \boldsymbol{\theta}^{(t)})$
  Compute $\mathcal{A}(\boldsymbol{\delta}^{(t)}) = \{k \geq 2 : \delta_{k1}^{(t)} < \sum_{j=2}^{q} \max(0, -\delta_{kj}^{(t)})\}$
  for $k = 1$ to K **do**
    Update $\boldsymbol{\delta}_k^{(t+1)} =$

$$\begin{cases} \boldsymbol{\delta}_k^{(t)} - \gamma_t \left(\nabla_{\delta_k} L(\boldsymbol{\delta}^{(t)}, \boldsymbol{\theta}^{(t)}) + \lambda \nabla_{\delta_k} J(\boldsymbol{\delta}_k^{(t)})\right) & \text{for } k \in \mathcal{A}(\boldsymbol{\delta}^{(t)}) \\ \boldsymbol{\delta}_k^{(t)} - \gamma_t \nabla_{\delta_k} L(\boldsymbol{\delta}^{(t)}, \boldsymbol{\theta}^{(t)}) & \text{for } k \notin \mathcal{A}(\boldsymbol{\delta}^{(t)}) \end{cases}$$

  end for
end for

General penalization methods for fitting neural network model with constraints have appeared in several papers (Platt and Barr 1988; Lillo et al. 1993; Xia and Wang 2004). These works aim to solve the optimization problems with the gradient descent method for efficient computation. On the basis of existing studies of general penalization methods, we propose a novel application as a specialized version of the penalization methods to the NMQN model.

Theorem 1 shows that we can obtain the solution of Equation (5) by solving Equation (12). The proof of Theorem 1 is appended in the supplementary material. As we do not use the barrier function in Algorithm 1, it is unnecessary to consider an appropriate choice of M and ϵ in $B(\boldsymbol{\delta}; M)$.

Theorem 1. Let $(\hat{\boldsymbol{\delta}}, \hat{\boldsymbol{\theta}})$ be the optimal solution of Equation (12) and $\lambda_{\max} = \sup_{\delta \in \mathbb{R}^{qK}, \theta \in \Theta} \|\nabla_{\delta} L(\boldsymbol{\delta}, \boldsymbol{\theta})\|_{\infty}$, where $\|\cdot\|_{\infty}$ is the infinite norm. Then $\hat{\delta}_{k1} = \max(\hat{\delta}_{k1}, \sum_{j=2}^{q} \max(0, -\hat{\delta}_{kj}))$ for $\lambda > \lambda_{\max}$, which implies that the solution of Equation (12) is equal to the solution of Equation (5).

Once we solve (12) to have the solution $(\hat{\boldsymbol{\delta}}, \hat{\boldsymbol{\theta}})$, we let the estimated τ_k-quantile function of the NMQN as

$$\widehat{f}_{\tau_k}(\mathbf{X}) = \sum_{l=1}^{k} \mathbf{z}(\mathbf{X}; \widehat{\boldsymbol{\theta}})^{\top} \widehat{\boldsymbol{\delta}}_l. \tag{13}$$

Because the gradient of $\rho_{\tau}(\cdot)$ is bounded by 1 for all $\tau \in (0, 1)$ and $\mathbf{z}(\mathbf{X}; \boldsymbol{\theta}) \in \{1\} \times [0, 1]^{q-1}$ for all $\mathbf{X} \in \mathbb{R}^p$ and $\boldsymbol{\theta} \in \Theta$, we can easily choose a $\lambda > \lambda_{\max} = 1$.

For a comprehensive understanding of the l_1-penalization algorithm, we compare the trajectories of the updated solutions of the interior point algorithm and the l_1-penalization algorithm with the following toy example:

$$\begin{aligned} \min_{\delta \in \mathbb{R}^2} \quad & (\delta_1 + 1)^2 + \frac{1}{4}(\delta_2 + 5)^2 \\ \text{subject to} \quad & \delta_1 - \max(0, -\delta_2) \geq 0. \end{aligned} \tag{14}$$

Figures 1(a) and 1(b) illustrate the solutions of the interior point and l_1-penalization algorithms respectively. In the interior point algorithm, $\epsilon = 10^{-4}$ and $M = 10^6$ are used in the barrier function. In both figures, the hollow circle denotes the initial solution; the gray points denote the updated solutions; the cross indicates the optimal solution of this example; and the shaded area denotes the feasible set. As illustrated in Figure 1(a), the solution obtained by the interior point algorithm becomes stuck at the boundary of the feasible region after a few iterations.

The l_1-penalization algorithm reaches the optimal solution as shown in Figure 1(b). Figures 2(a) and 2(b) are close-up views of the rectangles in Figures 1(a) and 1(b), which help to visualize the trajectories of the updated solutions. Vector v_1 in Figures 1(a) and 1(b) denotes the negative gradient of the objective function in (14); and v_2 in Figure 2(a) denotes the negative gradient of the barrier function used in the interior point algorithm; v_2 in Figure 2(b) denotes the negative gradient of the l_1-penalty function of the l_1-penalization algorithm. The norm of v_2 in Figure 2(a) is too small such that the solution cannot be updated any more without correcting the learning rate or the parameters in the barrier function, because the barrier function is not defined at the next step. However, the l_1-penalization algorithm updates the current solution along v_3 even when the current solution is not contained in the feasible region.

4.3. Non-crossing Quantile Estimates

Theorem 1 shows that the estimated quantile functions defined by Equation (13) are noncrossing, but it does not imply that we can directly obtain the noncrossing estimates using the l_1-penalization algorithm. Let $(\hat{\boldsymbol{\theta}}^{(T)}, \hat{\boldsymbol{\delta}}^{(T)})$ be the solution obtained from Algorithm 1. Even if we set $\lambda > \lambda_{\max}$, there is a chance of obtaining $\hat{\delta}_{k1}^{(T)} \neq \max(\hat{\delta}_{k1}^{(T)}, \sum_{j=2}^{q} \max(0, -\hat{\delta}_{kj}^{(T)}))$ for some k, because the objective function of Equation (12) is nonconvex as well as nondifferentiable. In this case, we can apply a simple correction by replacing $\hat{\delta}_{k1}^{(T)}$ with $\max\left(\hat{\delta}_{k1}^{(T)}, \sum_{j=2}^{q} \max(0, -\hat{\delta}_{kj}^{(T)})\right)$ for all k, as discussed in Section 4.1. However, the value of the objective function can significantly increase from only this simple correction. To effectively prevent this undesirable result, we use the post-correction method that solves Equation (15),

$$\min_{\delta \in \mathbb{R}^{qK}} \quad L(\boldsymbol{\delta}, \hat{\boldsymbol{\theta}}^{(T)}) + \lambda \sum_{k=2}^{K} J(\boldsymbol{\delta}_k), \tag{15}$$

by the subgradient method. Theorem 2 shows that the solution obtained by the subgradient method converges. The proof is appended in the supplementary material. In practice, we terminate the subgradient iteration when the number of iterations

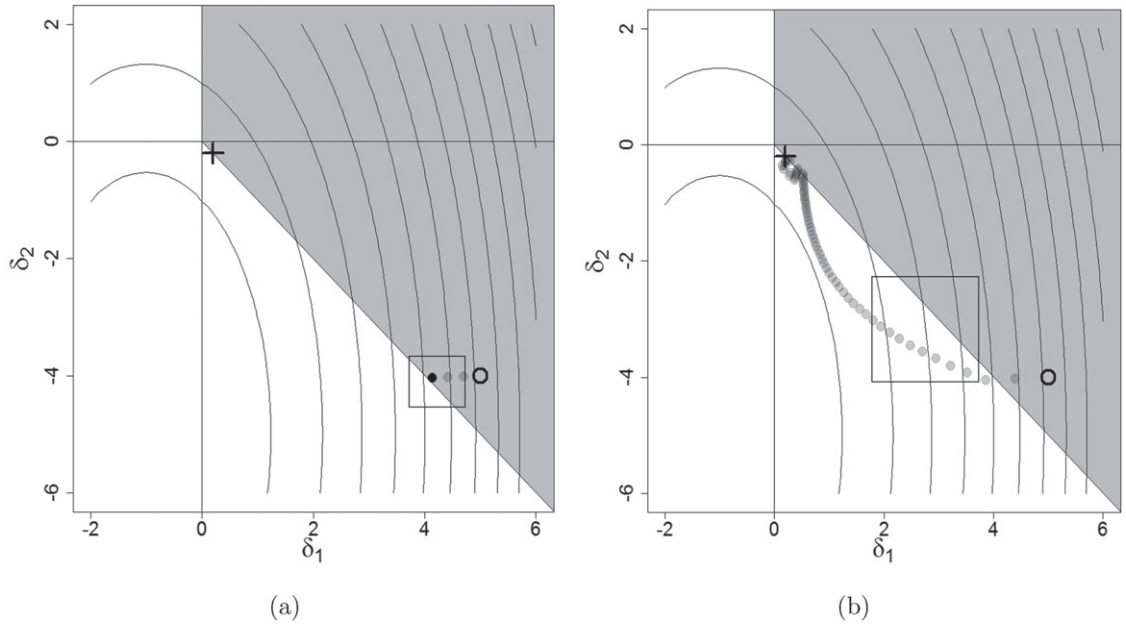


Figure 1. Left: The updated solutions by the interior point algorithm; Right: The updated solutions by the l_1-penalization algorithm.

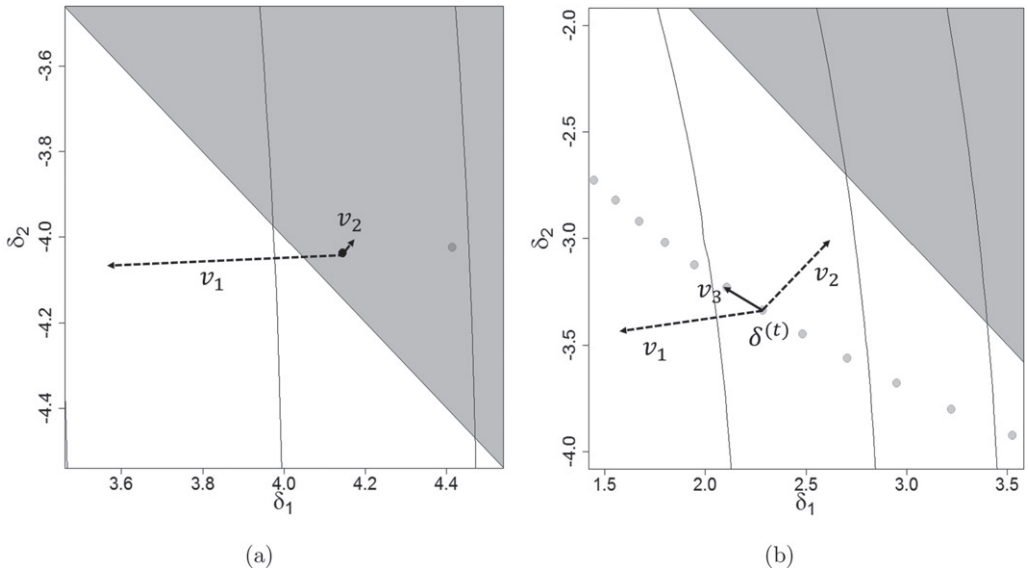


Figure 2. Left: The negative gradient vectors of the objective function and the barrier function of (14) in the interior point method; Right: those of the objective function and the l_1-penalty function in the l_1-penalization algorithm.

exceeds a predetermined threshold and the solution is contained in $\mathcal{C}_d$.

Theorem 2. Let $\tilde{\boldsymbol{\delta}}$ be the solution of Equation (15) and let the solution at the sth step of the subgradient method by $\tilde{\boldsymbol{\delta}}^{(s)}$. If the learning rate of the subgradient method is given by $\gamma_s = 1/s$, then

$$\lim_{s\to\infty}\left(L(\tilde{\boldsymbol{\delta}}^{(s)},\hat{\boldsymbol{\theta}}^{(T)})+\lambda\sum_{k=2}^{K}J(\tilde{\delta}_k^{(s)})\right)=L(\tilde{\boldsymbol{\delta}},\hat{\boldsymbol{\theta}}^{(T)})+\lambda\sum_{k=2}^{K}J(\tilde{\delta}_k).$$

4.4. Related Work

The number of studies on quantile function estimation has been increasing in recent years in the field of neural network research. Wen et al. (2017) propose quantile estimates in sequence-to-sequence neural networks and Fan et al. (2019) applied quantile estimates to the attention model (Vaswani et al. 2017) in deep learning. These two works commonly use quantile functions for probabilistic forecasting. However, the crossing-problem of quantiles is not considered in these studies.

The first noncrossing quantile regression model with neural networks is MCQRNN (Cannon 2018). The quantile functions of MCQRNN share all parameters except the bias parameters in the input layer. MCQRNN sets arbitrarily chosen constants $0 < d_1 < \cdots < d_K$. The bias parameter of the jth node in the input layer of the τ_k-quantile function is denoted by b_{kj}. Then, the bias is given by $b_{kj} = b_j + c_j d_k$ for $b_j, c_j \in \mathbb{R}$ and $c_j > 0$. Consequently, $b_{1j} < \cdots < b_{Kj}$ for all j. Also, MCQRNN restricts all parameters in the hidden and the output layers to positive values, and it employs the monotone activation functions. Since the quantile functions are compositions of the layers, the noncrossing property is obtained from the monotonicity of each layer.

Gasthaus et al. (2019) proposed the spline quantile function with recurrent neural networks (SQF-RNN) which directly models the conditional quantile function by spline basis functions. Unlike the previous works of Salinas et al. (2019) and Mukherjee et al. (2018), the SQF-RNN gives an explicit form of the conditional quantile function represented by the monotone spline regression model with varying coefficients (Hastie and Tibshirani 1993). Since monotone spline basis functions and positive valued varying coefficients are used, the noncrossing property of the conditional quantiles is achieved by the monotonicity of the spline regression model.

We employ linear constraints in NMQN. As mentioned earlier, this type of linear constraint is used by Bondell, Reich, and Wang (2010) and Takeuchi and Furuhashi (2004). Interestingly, minimizing the sum of the quantile losses in the NMQN is asymptotically equivalent to minimizing the continuous ranked probability score used in the SQF-RNN, which is promising for modeling the full conditional distribution using neural networks. However, the SQF-RNN requires $O(nK^2)$ computations in its output layer, which is given by $O(nK)$ in NMQN as well as MCQRNN.

It is easier to extend NMQN to a wider class of neural network models compared to MCQRNN and SQF-RNN because NMQN only requires the bounded condition of the hidden layer at the top. Just by adding the l_1-penalty function, we can reuse existing developed neural network models that contain convolution or recurrent layers. Based on the well-established theoretical results of convergence in the l_1-penalization algorithm, we can extend our optimization method to other constrained neural network models.

5. Simulation and Real Data Analysis

Using the predictive performance as a metric, we compare the estimated NMQN obtained by the interior point algorithm, the projected gradient algorithm, and the l_1-penalization algorithm. The proposed algorithms are implemented by R with tensorflow 1.14.0 on Ubuntu 16.04.6 LTS (64 bit). We denote the three algorithms as IP, PG, and l_1-P, respectively. We also present the

performance of MCQRNN (Cannon 2018) and the rearrange method (RM) (Chernozhukov, Fernández-Val, and Galichon 2010) combined with QRNN (Taylor 2000). MCQRNN is fitted by the "qrnn" package (Cannon 2011, 2018), and its weight parameters are regularized by the ridge penalty function. RM is implemented by ordering the outputs of QRNN fitted by the "qrnn' package. Details of the simulation environments are appended in the supplementary material. For all of the models that are considered, we select the optimal models by choosing the number of layers and nodes and regularization parameters of the neural networks with validation data. We report the training and test quantile risk in parallel to monitor overfitting.

5.1. Simulation

We describe the generative models for the simulations used in this section. We denote the predictor and the response variable by $\mathbf{X}$ and y.

- (M1) $y = \sin(\pi x_1)/(\pi x_1) + \epsilon$, where $\mathbf{X} = (x_1, \ldots, x_p) \sim U(-1, 1)^p$ and $\epsilon \sim N(0, \exp(1 - x_1)/10)$.
- (M2) $y = -\sum_{j=1}^{p} jx_j + \epsilon$, $\mathbf{X} = (x_1, \ldots, x_p) \sim U(-2, 2)^p$ and $\epsilon \sim N(0, \exp(1 + \min(\|\mathbf{X}\|_2^2 I(\|\mathbf{X}\|_2^2 \geq 1), 4)/2)$.
- (M3) $y = -3x + \epsilon$, $x \sim U(0, 4)$, $\epsilon \sim N(0, \exp(1 - x))$.

M1 has been used by Takeuchi et al. (2006). The true conditional τ-quantile function for a given predictor $\mathbf{X}$ is $f_\tau(\mathbf{X}) = \sin(\pi x_1)/(\pi x_1) + \sqrt{0.1 \exp(1 - x_1)}\Phi^{-1}(\tau)$, where $\Phi^{-1}(\cdot)$ is the quantile function of the standard normal distribution. M2 is a variant of M1 where the error variance depends on the norm of the predictor $\mathbf{X}$. As shown in Figure 3, the τ-quantile functions of M2 are nonlinear even though the conditional expectation function is linear. M3 is a simple linear regression model with heteroskedasticity.

The performance of the five noncrossing multiple quantile models (RM, MCQRNN, IP, PG, and l_1-P) is measured by averaging the training and test quantile risks, and computation time obtained from 100 simulations. We append additional results of predictive performance computed by different measures in the supplementary material. Throughout this article, we always set $\lambda = 5$ of the l_1-penalization algorithm. All weights and biases in the neural networks are initialized with random samples from the standard normal distribution, and the learning rate and the number of epochs are fixed to 0.005 and 2000. Note that the number of trials necessary to correct the learning rate in IP is not included when counting epochs.

First, Simulation 1 assumes M1. We set the target quantiles as $(\tau_1, \tau_2, \tau_3, \tau_4, \tau_5) = (0.10, 0.25,$ 0.50, 0.75, 0.90) and independently draw 200 samples for the training and validation data and 1000 samples for the test data. Table 1 shows the summary of the results of Simulation 1, which includes the mean and standard deviation (in parenthesis) of the quantile risks and those of the computation time required to fit an optimal model. When $p = 1$, the NMQN estimated by l_1-P is superior to or competitive with the others. IP shows poor performance in terms of both computation time and prediction. IP is frequently stuck around the boundary of the feasible set in M1. PG is competitive with l_1-P, but its computational cost is higher than that of l_1-P.

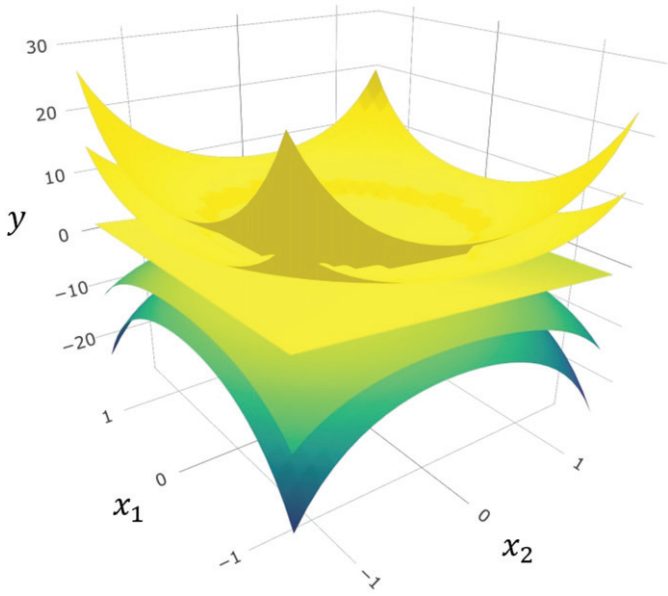


Figure 3. Illustration of the conditional quantile functions for $p = 2$ in Simulation 2.

When $p = 100$ and the irrelevant predictors are included, the estimated NMQN by l_1-P is the best in terms of predictive performance. MCQRNN requires very short computation time to obtain the estimated model, but it shows the worst performance among the five models. RM exhibits poor predictive performance, unlike when $p = 1$. Note that the performance of RM depends on that of QRNN. In addition, RM requires a sorting operation to produce noncrossing quantile estimates so that the computational cost increases for obtaining an optimal model. IP still requires high computational cost but PG does not.

We illustrate typical examples of estimated models in Figure 4. We generate 1000 training samples from M1 with $p = 1$ and let $K = 9$ with $(\tau_1, \ldots, \tau_9) = (0.1, \ldots, 0.9)$. Note that we display the result of QRNN instead of RM to investigate the multiple quantile estimates by neural networks. It is evident

Table 1. Comparison of five models in Simulation 1.

Performance measure	p	RM	MCQRNN	IP	PG	l_1-P
Training	1	0.087(0.006)	0.090(0.008)	0.155(0.02)	0.089(0.006)	0.088(0.006)
Test		0.090(0.003)	0.092(0.005)	0.155(0.02)	0.092(0.003)	0.091(0.003)
Time		21.25(9.47)	26.89(16.58)	3388.9(2018.5)	4043.3(2420)	5.90(1.57)
Training	100	0.162(0.01)	0.186(0.01)	0.165(0.02)	0.132(0.01)	0.132(0.01)
Test		0.163(0.006)	0.188(0.005)	0.168(0.02)	0.134(0.003)	0.134(0.003)
Time		27.01(9.67)	1.98(0.42)	3411.1(1988.8)	95.31(51.2)	5.93(1.51)

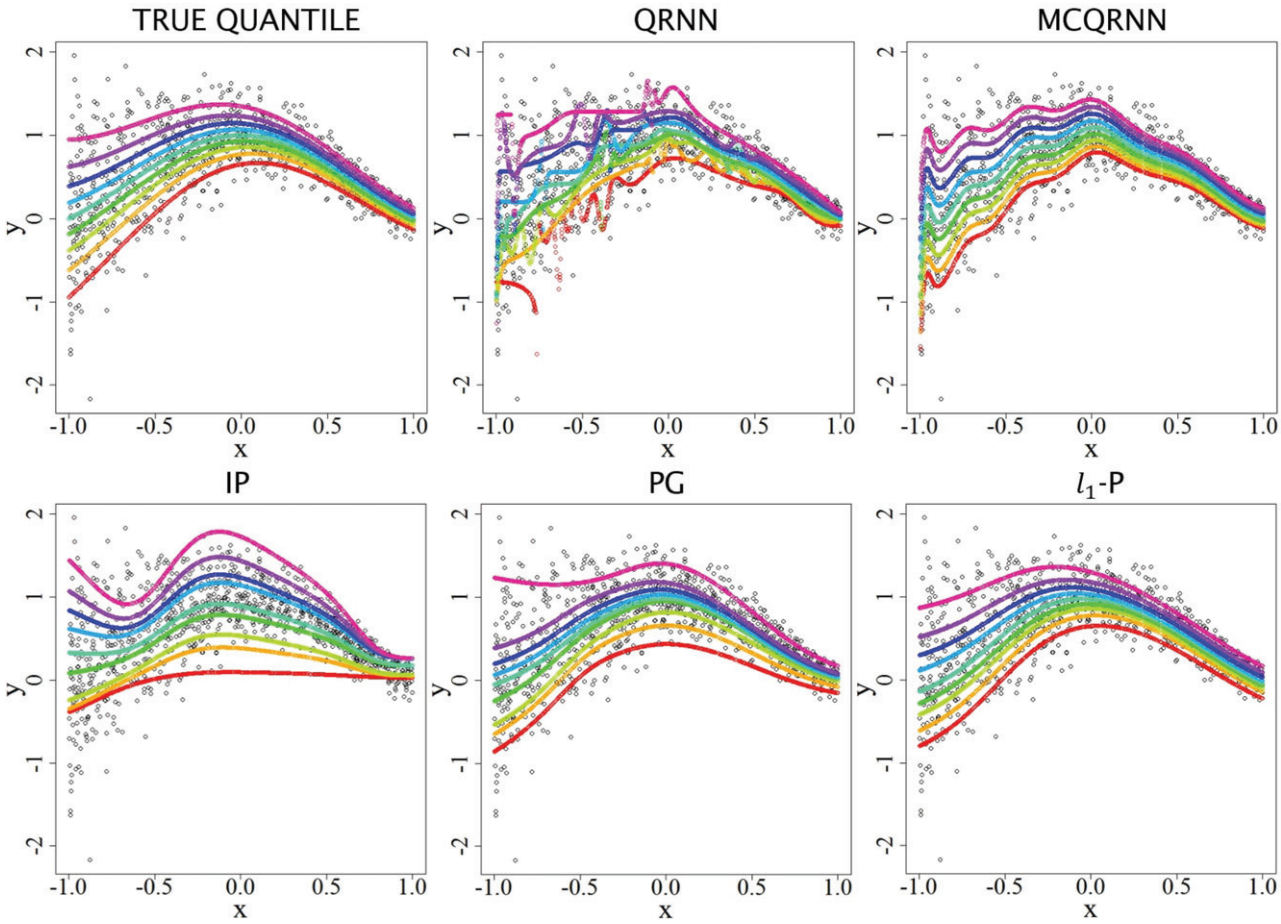


Figure 4. Examples of the fitted models for $(\tau_1, \ldots, \tau_9) = (0.1, \ldots, 0.9)$ under M1 with $p = 1$.

Table 2. Comparison of five models in Simulation 2.

Performance measure	p	RM	MCQRNN	IP	PG	l_1-P
Training	3	4.414(0.082)	4.288(0.043)	4.561(0.054)	4.585(0.04)	4.297(0.098)
Test		4.418(0.157)	4.295(0.134)	4.56(0.158)	4.585(0.147)	4.301(0.162)
Time		40.77(5.58)	1785.8(316.6)	79.2(73.3)	48.0(39.5)	19.57(1.38)
Training	5	5.866(0.06)	5.266(0.038)	5.782(0.113)	5.504(0.195)	5.302(0.062)
Test		6.025(0.18)	5.381(0.703)	5.805(0.198)	5.528(0.232)	5.348(0.128)
Time		40.68(6.62)	1403.9(573.3)	65.5(2.7)	380.9(324.9)	20.06(1.7)

that the estimated conditional quantile functions of QRNN are crossing in Figure 4. The high and low conditional quantile functions seem to be overfitted, while the median function is relatively well-estimated. This result implies that we should separately select the tuning parameters of each quantile function to improve the estimated model. MCQRNN solves the crossing problem of QRNN and provides a relatively well-fitted model for multiple quantiles. The NMQNs fitted by IP, PG, and l_1-P show similar quantile patterns, but IP seems to fail to estimate the 0.1-conditional quantile function.

Simulation 2 assumes M2 for $p = 3$ and 5. The training, validation, and test data consist of 10,000 random samples drawn from M2. Table 2 presents a summary of Simulation 2, including the mean and standard deviation (in parentheses) of the quantile risks and the computation times. There are no significant differences in predictive performance across all of the models. However, among the algorithms fitting NMQN, l_1-P outperforms all others in terms of both prediction and computation time. PG requires a long computation time to fit NMQN, while IP shows a comparable performance in computation time compared with l_1-P. We speculate that the solutions obtained by IP are less likely to be stuck in Simulation 2 compared to Simulation 1.

Compared to Simulation 1, the computation time of MCQRNN drastically increases in Simulation 2. Recall that MCQRNN is implemented by the "qrnn" package in R. The "qrnn" package internally uses the "nlm" package based on the second-order optimization methods such as the Newton method. Therefore, MCQRNN used in this paper is cannot be applied to large-scale data. If MCQRNN were fitted by the gradient descent method, then the computation time would be greatly reduced compared to that shown in Table 2, but the presented predictive performance is not guaranteed with the same number of epochs used in our simulations.

Table 3. Comparison of five models in Simulation 3.

Model	Performance Measure	The number of hidden layers				
		1	2	3	4	5
RM	Training	0.189(0.008)	0.192(0.008)	0.196(0.008)	0.202(0.011)	0.258(0.177)
	Test	0.189(0.009)	0.193(0.010)	0.197(0.010)	0.202(0.014)	0.257(0.177)
MCQRNN	Training	0.329(0.091)	0.332(0.195)	0.580(0.527)	1.019(0.588)	1.220(0.505)
	Test	0.329(0.091)	0.332(0.191)	0.582(0.531)	1.022(0.590)	1.219(0.506)
IP	Training	0.328(0.040)	0.344(0.036)	0.562(0.257)	0.928(0.150)	1.000(0.027)
	Test	0.328(0.040)	0.345(0.037)	0.563(0.259)	0.929(0.150)	1.000(0.029)
PG	Training	0.206(0.010)	0.205(0.01)	0.207(0.010)	0.213(0.013)	0.227(0.081)
	Test	0.206(0.011)	0.205(0.012)	0.207(0.011)	0.213(0.013)	0.227(0.078)
l_1-P	Training	0.202(0.011)	0.198(0.011)	0.201(0.010)	0.209(0.025)	0.228(0.098)
	Test	0.201(0.012)	0.198(0.012)	0.201(0.011)	0.210(0.026)	0.229(0.099)

Table 4. Sensitivity of ℓ_1-P for different λ's.

λ	0.01	0.1	1	3	5	10
Training	0.198(0.126)	0.088(0.006)	0.088(0.006)	0.089(0.008)	0.088(0.006)	0.088(0.006)
Test	0.199(0.126)	0.091(0.003)	0.091(0.003)	0.091(0.005)	0.091(0.003)	0.091(0.003)

Table 5. Comparison of six models using YVRprecip data.

Performance measure	RM	QRF	MCQRNN	IP	PG	l_1-P
Training	1.07(0.02)	1.15(0.02)	1.06(0.01)	1.35(0.1)	1.11(0.04)	1.07(0.01)
Test	1.08(0.03)	1.15(0.03)	1.07(0.02)	1.34(0.1)	1.13(0.04)	1.08(0.03)
Time	19.4(4.81)	5.02(0.14)	1104(102.5)	3473(2529.9)	2403(1708.7)	2.38(4.24)

Simulation 3 investigates the robustness of the estimation models to the choice of the number of hidden layers in neural networks. M3 is assumed and the numbers of training, validation, and test samples are fixed at 1000. In Simulation 3, the MCQRNN is implemented by tensorflow because the "qrnn" package can only be used to fit the MCQRNN with a maximum of two layers. Table 3 shows the predictive performance of the models according to the number of layers. RM with a single layer is best at prediction, and l_1-P and PG with a single layer are competitive with RM, while both MCQRNN and IP perform poorly. As the number of the layers increases, the predictive performance of MCQRNN and NMQN estimated by IP become worse. However, l_1-P and PG do not show a significant loss in predictive performance when more layers are added to the NMQN. Considering the results of l_1-P and PG, the failure of IP with multiple layers can be attributed to the convergence problem of this algorithm.

Finally, we investigate the sensitivity of the predictive performance of l_1-P to the selection of λ. Theoretically, $\lambda > 1$ is sufficient to obtain the noncrossing conditional quantile functions in our model. We assume M1 with $p = 1$ and reuse the experiment design of Simulation 1. NMQN is fitted by l_1-P with various λs. Table 4 shows the quantile risks attained from the simulation repeated 20 times. It can be seen that there is no significant difference in predictive performance for $\lambda > 1$, which is consistent with Theorem 1.

5.2. Real Data Analysis

We apply NMQN with l_1-P to fit YVRprecip data (Cannon 2018), Energy data (Candanedo, Feldheim, and Deramaix 2017), and Beijing PM2.5 data (Liang et al. 2015). The YVRprecip dataset consists of the daily precipitation totals at Vancouver International Airport from 1971 to 2000. The YVRprecip dataset has 10, 958 observations; we use 6136 randomly sampled observations as the training dataset, 1534 observations as the validation dataset, and the remainder as the test dataset. Each observation consists of four variables denoting seasonal cycle and daily sea-level pressure, 700-hPa specific humidity, and 500-hPa geopotential height (see Cannon 2018 for more details). In this section, we present the results of our analysis only for YVRprecip data. The descriptions and results of the other datasets are appended in the supplementary materials.

We set the target quantiles as $(\tau_1, \tau_2, ..., \tau_9) = (0.1, 0.2, ..., 0.9)$ and compare RM, MCQRNN, IP, PG, and l_1-P. An optimal model is chosen by the regularization used in Section 5.1. Additionally, we consider the quantile regression forests (QRF) (Meinshausen 2006). We choose an optimal model by selecting the tree depth of the QRF. The predictive performance and computation time for each model are measured by simulation, repeated 20 times. The mean and standard deviation of the quantile risk and the computation time are shown in Table 5.

Table 5 indicates that QRF requires the least amount of computing time compared to the others. However, its predictive performance is the worst among all of the models considered. RM and l_1-P are best in terms of predictive performance. Although MCQRNN is competitive with l_1-P in terms of prediction, MCQRNN takes a longer time to fit the model than l_1-P. The predictive performance of PG is similar to that of l_1-P, but it takes the longest time because it may require a

large number of computations for its projection step. IP does not require much time for the estimation. Judging from the predictive performance of IP, it may not be able to update the solution at the boundary of the feasible region.

6. Concluding Remarks

We propose a neural network model for predicting multiple conditional quantiles that satisfies the noncrossing property. By reparameterization of the model coefficients, we develop a scalable estimation method and also propose an efficient computation algorithm based on the first-order optimization. The developed NMQN can be viewed as a generalized additive model in which the basis functions are constructed by the fully connected neural network. Unlike MCQRNN, the noncrossing property of the NMQN is derived only from the bounded condition of the hidden layers such that the modeling of the feature function $\mathbf{z}(\cdot;\boldsymbol{\theta})$ is flexible. As an example, NMQN can easily be extended to the noncrossing quantile regression on spatial predictors using a convolution neural network (Krizhevsky, Sutskever, and Hinton 2012).

Additionally, employing the l_1-penalization algorithm to fit NMQN provides a useful framework for the first-order optimization method with inequality constraints. By introducing a l_1-penalty function, the l_1-penalization algorithm efficiently produces a nearly optimal solution of NMQN. Furthermore, as the solution is updated only by the gradient method, it is easy to extend to large-scale problems. The simulation and analysis of real data showed that the performance of NMQN is better than or competitive with that of existing methods.

Using inequality constraints, we can structurally restrict a model space that may be desired for model interpretability. Computation on the associated feasible set is one of the main problems in fitting complex models such as neural networks. The l_1-penalization algorithm suggests an optimization method implemented by the gradient descent algorithm on a feasible set induced by linear constraints. We expect that the concept of the l_1-penalization algorithm can be extended to more general types of constraints. However, little is known of the convergence property of the l_1-penalization algorithm for nonconvex objective functions. For example, when we used the ridge penalty instead of l_1-penalty, the performance of the estimated model became worse. We conjecture that the nondifferentiability of the l_1-penalty function is closely related to the convergence property of the algorithm. Thus, further investigations of the convergence properties of the l_1-penalization algorithm are necessary in future.

Supplemental Materials

Code: The R code used to fit NMQN by the l_1-penalization algorithm are provided in ("code_submit.zip").

Supp: Additional notes for proposition and proofs of theorems are provided in ("supp.pdf").

Funding

J. Jeon was partly supported by the National Research Foundation of Korea(NRF) grant funded by the Korea government(MSIT) (2019R1F1A1059011) and Basic Science Research Program through the National Research Foundation of Korea (NRF) funded by the Ministry of Education (2018R1A6A1A06024977). J. Lee was supported by Basic Science Research Program through the National Research Foundation of Korea (NRF) funded by the Ministry of Education (2018R1A6A1A06024977). Y. Kim was supported by the National Research Foundation of Korea(NRF) grant funded by the Korea government(MSIT) (No. 2020R1A2C3A01003550).

References

Alber, Y. I., Iusem, A. N., and Solodov, M. V. (1998), "On the Projected Subgradient Method for Nonsmooth Convex Optimization in a Hilbert Space," *Mathematical Programming*, 81, 23–35. [1240]

Bondell, H. D., Reich, B. J., and Wang, H. (2010), "Noncrossing Quantile Regression Curve Estimation," *Biometrika*, 97, 825–838. [1238,1239,1243]

Candanedo, L. M., Feldheim, V., and Deramaix, D. (2017), "Data Driven Prediction Models of Energy Use of Appliances in a Low-energy House," *Energy and Buildings*, 140, 81–97. [1246]

Cannon, A. J. (2011), "Quantile Regression Neural Networks: Implementation in r and Application to Precipitation Downscaling," *Computers & Geosciences*, 37, 1277–1284. [1244]

——— (2018), "Non-crossing Nonlinear Regression Quantiles by Monotone Composite Quantile Regression Neural Network, With Application to Rainfall Extremes," *Stochastic Environmental Research and Risk Assessment*. [1238,1243,1244,1246]

Chernozhukov, V., Fernández-Val, I., and Galichon, A. (2010), "Quantile and Probability Curves Without Crossing," *Econometrica*, 78, 1093–1125. [1244]

Chorowski, J., and Zurada, J. M. (2014), "Learning Understandable Neural Networks With Nonnegative Weight Constraints," *IEEE Transactions on Neural Networks and Learning Systems*, 26, 62–69. [1240]

Doksum, K., and Koo, J.-Y. (2000), "On Spline Estimators and Prediction Intervals in Nonparametric Regression," *Computational Statistics & Data Analysis*, 35, 67–82. [1238]

Fan, C., Zhang, Y., Pan, Y., Li, X., Zhang, C., Yuan, R., Wu, D., Wang, W., Pei, J., and Huang, H. (2019), "Multi-horizon Time Series Forecasting With Temporal Attention Learning," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, Anchorage, AK, USA*, pp. 2527–2535. [1243]

Feng, Y., Li, R., Sudjianto, A., and Zhang, Y. (2010), "Robust Neural Network With Applications to Credit Portfolio Data Analysis," *Statistics and Its Interface*, 3, 437. [1238]

Gasthaus, J., Benidis, K., Wang, Y., Rangapuram, S. S., Salinas, D., Flunkert, V., and Januschowski, T. (2019), "Probabilistic Forecasting With Spline Quantile Function RNNs," in *The 22nd International Conference on Artificial Intelligence and Statistics, Naha, Okinawa, Japan*, pp. 1901–1910. [1238,1243]

Hall, P., Wolff, R. C., and Yao, Q. (1999), "Methods for Estimating a Conditional Distribution Function," *Journal of the American Statistical Association*, 94, 154–163. [1238]

Hastie, T., and Tibshirani, R. (1993), "Varying-coefficient Models," *Journal of the Royal Statistical Society*, Series B, 55, 757–779. [1243]

Hatalis, K., Lamadrid, A. J., Scheinberg, K., and Kishore, S. (2017), "Smooth Pinball Neural Network for Probabilistic Forecasting of Wind Power," arXiv preprint arXiv:1710.01720. [1238]

Hwang, C., and Shim, J. (2005), "A Simple Quantile Regression Via Support Vector Machine," In *International Conference on Natural Computation*, 512–520. Springer. [1238]

Iusem, A. N. (2003), "On the Convergence Properties of the Projected Gradient Method for Convex Optimization," *Computational & Applied Mathematics*, 22, 37–52. [1240]

Kai, B., Li, R., and Zou, H. (2010), "Local Composite Quantile Regression Smoothing: An Efficient and Safe Alternative to Local Polynomial Regression," *Journal of the Royal Statistical Society*, Series B, 72, 49–69. [1238]

Koenker, R. (2004), "Quantile Regression for Longitudinal Data," *Journal of Multivariate Analysis*, 91, 74–89. *https://doi.org/10.1016/j.jmva.2004.05.006* [1238]

Koenker, R., and Bassett Jr, G. (1978), "Regression Quantiles," *Econometrica: Journal of the Econometric Society*, 46, 33–50. [1238]

Koenker, R., Ng, P., and Portnoy, S. (1994), "Quantile Smoothing Splines," *Biometrika*, 81, 673–680. [1238,1239]

Krizhevsky, A., Sutskever, I., and Hinton, G. E. (2012), "Imagenet Classification With Deep Convolutional Neural Networks," in *Advances in Neural Information Processing Systems*, pp. 1097–1105. [1247]

Li, Y., and Zhu, J. (2008), "L1-norm Quantile Regression," *Journal of Computational and Graphical Statistics*, 17, 163–185. [1238]

Liang, X., Zou, T., Guo, B., Li, S., Zhang, H., Zhang, S., Huang, H., and Chen, S. X. (2015), "Assessing Beijing's pm2. 5 Pollution: Severity, Weather Impact, Apec and Winter Heating," *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 471, 20150257. [1246]

Lillo, W. E., Loh, M. H., Hui, S., and Zak, S. H. (1993), "On Solving Constrained Optimization Problems With Neural Networks: A Penalty Method Approach," *IEEE Transactions on Neural Networks*, 4, 931–940. [1241]

Meinshausen, N. (2006), "Quantile Regression Forests," *Journal of Machine Learning Research*, 7, 983–999. [1238,1246]

Mukherjee, S., Shankar, D., Ghosh, A., Tathawadekar, N., Kompalli, P., Sarawagi, S., and Chaudhury, K. (2018), "Armdn: Associative and Recurrent Mixture Density Networks for Eretail Demand Forecasting," arXiv preprint arXiv:1803.03800. [1243]

Pathak, D., Krahenbuhl, P., and Darrell, T. (2015), "Constrained Convolutional Neural Networks for Weakly Supervised Segmentation," in *Proceedings of the IEEE International Conference on Computer Vision*, pp. 1796–1804. [1240]

Platt, J. C., and Barr, A. H. (1988), "Constrained Differential Optimization," in *Neural Information Processing Systems*, Pasadena: California Institute of Technology, 612–621. [1241]

Salinas, D., Flunkert, V., Gasthaus, J., and Januschowski, T. (2019), "Deepar: Probabilistic Forecasting With Autoregressive Recurrent Networks," *International Journal of Forecasting*, 36, 1181–1191. [1243]

Sangnier, M., Fercoq, O., and d'Alché Buc, F. (2016), "Joint Quantile Regression in Vector-valued RKHSS," in *Advances in Neural Information Processing Systems*, pp. 3693–3701. [1238]

Silverman, B. W. (1985), "Some Aspects of the Spline Smoothing Approach to Non-parametric Regression Curve Fitting," *Journal of the Royal Statistical Society*, Series B, 47, 1–52. [1238]

Sohn, I., Kim, S., Hwang, C., and Lee, J. W. (2008), "New Normalization Methods Using Support Vector Machine Quantile Regression Approach in Microarray Analysis," *Computational Statistics & Data Analysis*, 52, 4104–4115. [1238]

Takeuchi, I. and Furuhashi, T. (2004), "Non-crossing Quantile Regressions by SVM," in *2004 IEEE International Joint Conference on Neural Networks, 2004. Proceedings*, 1, 401–406. IEEE. [1238,1243]

Takeuchi, I., Le, Q. V., Sears, T. D., and Smola, A. J. (2006), "Nonparametric Quantile Estimation," *Journal of Machine Learning Research*, 7, 1231–1264. [1238,1244]

Taylor, J. W. (2000), "A Quantile Regression Neural Network Approach to Estimating the Conditional Density of Multiperiod Returns," *Journal of Forecasting*, 19, 299–311. [1238,1244]

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017), "Attention is All You Need," in *Advances in Neural Information Processing Systems*, 5998–6008. [1243]

Wen, R., Torkkola, K., Narayanaswamy, B., and Madeka, D. (2017), "A Multi-horizon Quantile Recurrent Forecaster," arXiv preprint arXiv:1711.11053. [1243]

Xia, Y. (2004), "An Extended Projection Neural Network for Constrained Optimization," *Neural Computation*, 16, 863–883. [1240]

Xia, Y., and Wang, J. (2004), "A Recurrent Neural Network for Nonlinear Convex Optimization Subject to Nonlinear Inequality Constraints," *IEEE Transactions on Circuits and Systems I: Regular Papers*, 51, 1385–1394. [1241]

Xu, Q., Deng, K., Jiang, C., Sun, F., and Huang, X. (2017), "Composite Quantile Regression Neural Network With Applications," *Expert Systems with Applications*, 76, 129–139. [1238]

Yu, K., and Jones, M. (1998), "Local Linear Quantile Regression," *Journal of the American Statistical Association*, 93, 228–237. [1238]

Zou, H., Yuan, M. (2008a), "Composite Quantile Regression and the Oracle Model Selection Theory," *The Annals of Statistics*, 36, 1108–1126. [1239]

——— (2008b), "Regularized Simultaneous Model Selection in Multiple Quantiles Regression," *Computational Statistics & Data Analysis*, 52, 5296–5304. [1239]